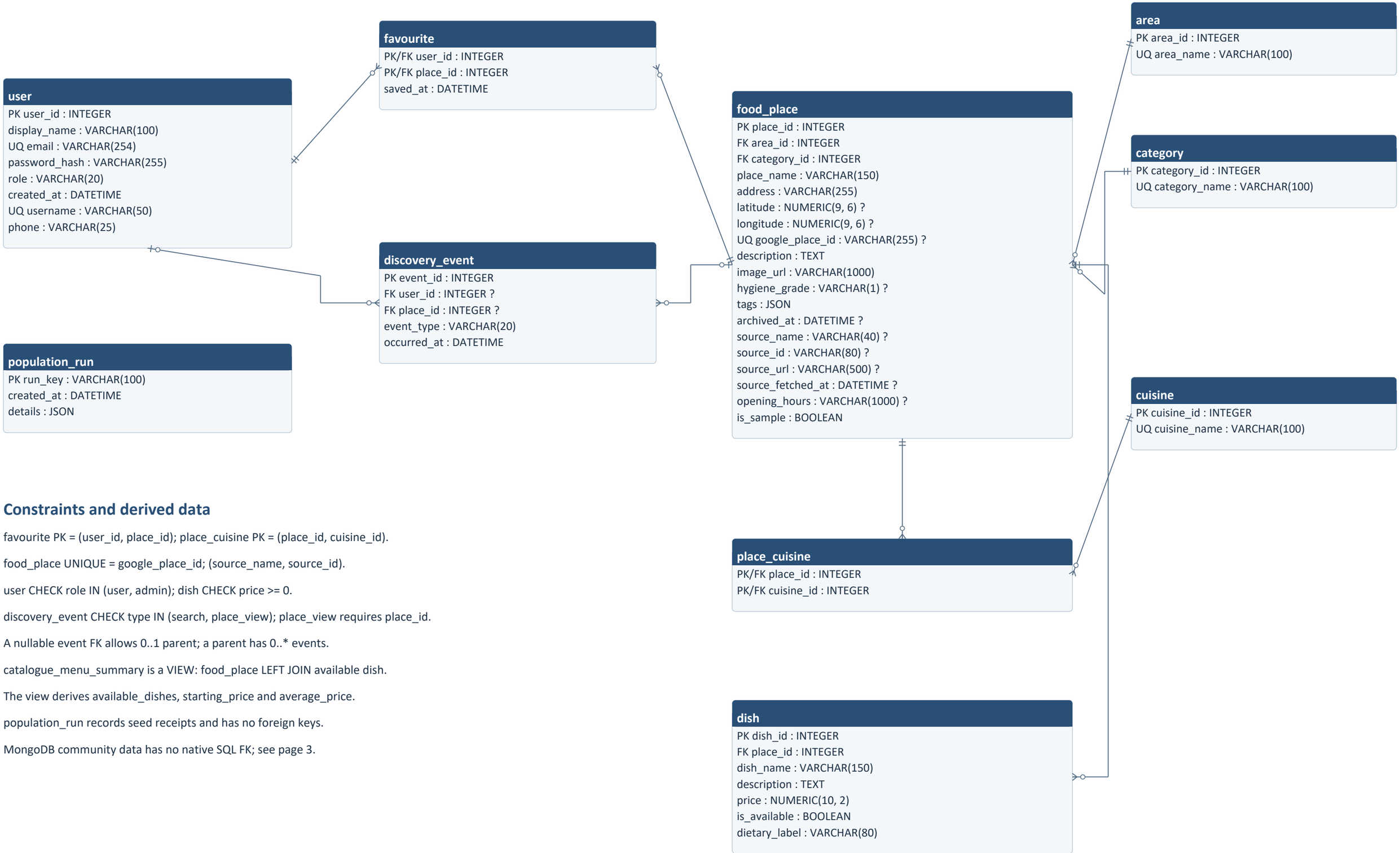
Active SQL schema

All 10 active tables and all declared attributes. ? = nullable; two PK fields form a composite key.



Constraints and derived data

favourite PK = (user_id, place_id); place_cuisine PK = (place_id, cuisine_id).

food_place UNIQUE = google_place_id; (source_name, source_id).

user CHECK role IN (user, admin); dish CHECK price >= 0.

discovery_event CHECK type IN (search, place_view); place_view requires place_id.

A nullable event FK allows 0..1 parent; a parent has 0..* events.

catalogue_menu_summary is a VIEW: food_place LEFT JOIN available dish.

The view derives available_dishes, starting_price and average_price.

population_run records seed receipts and has no foreign keys.

MongoDB community data has no native SQL FK; see page 3.

Source: backend/models.py, mongo.py, community.py, community_migration.py, population.py and database_lab.py

Retained SQL migration schema

Seven legacy tables remain in the code. Live community routes use MongoDB; these tables are migration inputs.



Additional declared foreign keys

content.author_id → user; content.removed_by_id → user (nullable).

content.place_id → food_place; content_dish.dish_id → dish.

upvote.user_id → user; report.reporter_id → user.

report.resolved_by_id → user (nullable).

Each subtype FK is (content_id, kind) → content(content_id, kind).

Subtype PK is content_id; each parent has 0..1 row per subtype.

Source: backend/models.py, mongo.py, community.py, community_migration.py, population.py and database_lab.py

MongoDB document model

Five collections; embedded dishes stay inside content. All cross-database references are validated by the API.

content

_id : INT/LONG > 0 (unique identity)

kind : review | thread | comment

author_id, place_id : INT/LONG > 0 → SQL user / food_place

body : STRING, length 1–5000

created_at, updated_at : DATE; removed_at : DATE or null

removed_by_id : INT/LONG or null → SQL user

is_demo : BOOL (optional)

dishes : ARRAY[0..20] of embedded objects

  dish_id : INT/LONG > 0 → SQL dish

  feedback : STRING (≤1000) or null

review requires title (≤150) + rating (INT 1–5); no thread_id

thread requires title; no rating or thread_id

comment requires thread_id → content(kind=thread); no rating

Same-place thread/dish and duplicate dishes checked by API.

Removal timestamp and remover must be present together.

Seeded: seed_key (STRING). Migrated: legacy_checksum.

Partial UNIQUE seed_key; indexes place_feed and thread_replies.

1 ← 0..*

upvotes

_id : ObjectId (automatically generated)

user_id : INT/LONG → SQL user; content_id : INT/LONG → content

voted_at : DATE; seeded votes additionally carry is_demo : BOOL

UNIQUE (user_id, content_id); separate content_id index

JSON Schema requires user_id, content_id, voted_at.

1 ← 0..*

reports

_id : INT/LONG > 0; content_id : INT/LONG → content

reporter_id : INT/LONG → SQL user; reason : STRING (1–1000)

status : open | resolved | dismissed; created_at : DATE

On resolution: resolved_by_id → SQL user, resolved_at, resolution_note.

Those resolution fields are written by the API, not in its JSON Schema.

Seeded: seed_key, is_demo; migrated: legacy_checksum.

Index (status, created_at); partial UNIQUE seed_key.

counters

_id : STRING namespace (content or reports); value : numeric

Atomic $inc allocates IDs; migration uses $max; gaps are permitted.

No collection JSON Schema validator is configured.

population_runs

_id : ObjectId; completed_at : DATE; summary : OBJECT

summary includes version, synthetic flag, before/after/added counts,

csvSha256 and completedAt. No JSON Schema validator is configured.

Reference and consistency rules

Reviews, threads and comments share one content collection; replies reference a thread in that collection.

Votes/reports grow independently and reference content; feedback is embedded for bounded atomic updates.

Author/place/dish identities are SQL references, not MongoDB foreign keys. Soft deletion retains referenced identities.

No distributed ACID transaction. Population and migration use stable keys and receipts/checksums for resumable work.

Source: backend/models.py, mongo.py, community.py, community_migration.py, population.py and database_lab.py